

CLOUD COMPUTING

Unit IV: Cloud Platforms and Cloud Applications

TY B.Tech AI & Data Science | Semester VI | End Semester Notes

Course Code: 2301315A

Syllabus Coverage (Unit IV — 8 hrs | CO4)

- Amazon Web Services (AWS): Components, Amazon SimpleDB, EC2, Amazon Storage System, Amazon DynamoDB
- Cloud Computing Applications: Healthcare (ECG Analysis), Biology (Protein Structure Prediction), Geosciences (Satellite Image Processing)
- Business & Consumer Applications: CRM and ERP, Social Networking
- Google Cloud Application: Google App Engine
- Overview of OpenStack Architecture

1. Amazon Web Services (AWS)

AWS is a comprehensive, evolving cloud computing platform by Amazon that provides on-demand IT resources over the internet on a pay-as-you-go basis. Instead of owning physical servers, organizations 'rent' resources from Amazon.

1.1 Key Characteristics of AWS

- On-Demand Delivery: Resources are available exactly when needed, with no upfront commitment.
- Scalability: Automatically scales from 100 to 1 million users without manual intervention.
- Pay-as-you-go: Users pay only for what they consume, like a utility bill.

1.2 AWS Global Infrastructure

AWS is deployed across a globally distributed physical infrastructure with three main levels:

Component	Description	Example
Regions	Physical locations worldwide containing multiple data centers	us-east-1 (N. Virginia), ap-south-1 (Mumbai)
Availability Zones (AZs)	Isolated data centers within a Region with redundant power/networking	Each region has 2–6 AZs
Edge Locations	Smaller CDN nodes used by CloudFront to cache content closer to users	Used for low-latency content delivery

□ **Note:** In the endsem, you may be asked to explain the 3-tier AWS infrastructure. Remember: Region > AZ > Edge Locations.

1.3 Core AWS Services and Components

AWS offers 200+ services. The four fundamental categories are Compute, Storage, Databases, and Networking.

A. Compute Services

- **Amazon EC2 (Elastic Compute Cloud): Resizable virtual servers (VMs) in the cloud. You choose the OS, RAM, and CPU.**
 - EC2 Instance: The actual virtual server (e.g., t3.micro, m5.large).
 - AMI (Amazon Machine Image): Blueprint/template containing the OS and configuration.
 - EBS (Elastic Block Store): Persistent 'hard drive' attached to the instance.
 - Security Groups: Virtual firewall that controls inbound/outbound traffic (port 80, 22, etc.).
 - Key Pairs: Public/private digital keys used for secure SSH login to the instance.
- **AWS Lambda: Serverless computing. Code runs only when triggered — no server management needed. Cost-effective for event-driven workloads.**
- **Amazon ECS/EKS: Services to run containerized applications (Docker) — ECS is managed containers; EKS is managed Kubernetes.**

B. Storage Services

- **Amazon S3 (Simple Storage Service): Object storage for files (images, videos, backups) stored in 'buckets'.**

- Buckets: Top-level containers (globally unique names).
 - Objects: Actual files stored inside a bucket.
 - Keys: Unique path/name for each object (e.g., images/photo.jpg).
 - Bucket Policies: JSON rules controlling access permissions.
 - Storage Classes: S3 Standard (frequent access), S3 Glacier (long-term archive, cheaper).
- **Amazon EBS (Elastic Block Store):** High-performance block storage attached to a single EC2 instance — like a virtual SSD.
 - **Amazon EFS (Elastic File System):** Shared file system mountable by hundreds of EC2 instances simultaneously, using NFS protocol. Multi-AZ resilience.
 - **Amazon FSx:** Specialized file systems — FSx for Windows (SMB), FSx for Lustre (HPC/AI), FSx for NetApp ONTAP (enterprise migrations).

C. Database Services

- **Amazon RDS (Relational Database Service):** Managed SQL databases (MySQL, PostgreSQL, Oracle, SQL Server). AWS handles patching and backups.
 - Read Replicas: Copies of DB to offload read traffic.
 - Multi-AZ Deployment: Standby DB copy in another AZ for high availability.
- **Amazon DynamoDB:** Serverless NoSQL key-value database. Single-digit millisecond performance at massive scale (10 trillion requests/day). Detailed in Section 3.
- **Amazon Aurora:** AWS proprietary relational DB — 5x faster than MySQL, 3x faster than PostgreSQL. Self-healing storage up to 128TB. Compatible with MySQL/PostgreSQL.
- **Amazon Redshift:** Data warehouse for analytics (petabyte-scale). Uses columnar storage for fast queries on large datasets. Best for Business Intelligence.
- **Amazon ElastiCache:** In-memory caching (Redis/Memcached) to speed up apps by storing frequently accessed data in RAM.

D. Networking & Content Delivery

- **Amazon VPC (Virtual Private Cloud):** Private, isolated section of the AWS cloud where you define IP addresses and subnets.
 - Subnets: IP range segments (Public subnets for web servers, Private subnets for databases).
 - Internet Gateway (IGW): 'Doorway' for VPC resources to connect to the internet.
 - Route Tables: Rules for directing network traffic.
 - NACL (Network Access Control List): Optional subnet-level firewall.
- **Amazon Route 53:** Scalable DNS web service — maps domain names (www.example.com) to IP addresses.

E. Security — AWS IAM (Identity and Access Management)

IAM is the 'gatekeeper' that controls who can access AWS resources and what they can do.

- **Users:** Individual identities (e.g., a developer) who log into the AWS Console.
- **Groups:** Collections of users (e.g., 'Admins') to apply permissions in bulk.
- **Roles:** Identities assumed by services (e.g., giving an EC2 server permission to access S3).
- **Policies:** JSON documents that explicitly Allow or Deny actions (e.g., Allow S3:GetObject).

2. Amazon SimpleDB

Amazon SimpleDB is a distributed, schema-less, non-relational database hosted on Amazon's cloud. It is written in Erlang and designed for simple querying via web service requests. It automatically indexes all stored data, enabling fast retrieval without manual index creation.

2.1 Core Data Model

SimpleDB Concept	Relational DB Equivalent	Description
Domain	Table	Top-level container for data items.
Item	Row / Record	A collection of attribute-value pairs. Each item can have up to 256 attributes and is uniquely identified by an item name.
Attribute	Column / Field	A key-value pair associated with an item. Flexible — each item can have different attributes.
Value	Cell data	Actual data stored. One attribute can have multiple values (multi-valued attributes).

2.2 Key Characteristics

- Schema-less Flexibility: Add new attributes to any item at any time — no table migration required.
- Automatic Indexing: Every attribute is indexed by default, enabling immediate queries on any field.
- Eventual Consistency (Default): Provides high read performance but data may not be immediately up-to-date everywhere. Supports Consistent Reads if required.
- High Availability: Data is automatically replicated across multiple Availability Zones.
- Simple API: Small set of API calls — PutAttributes, GetAttributes, Select, DeleteAttributes.

2.3 Features

- Flexibility: Easy addition of new attributes without predefined data formats.
- Efficiency: Quick data storage and retrieval.
- Scalability: New domains can be created to handle data volume increases (horizontal scaling).
- Easy Integration: Works seamlessly with other AWS services like EC2 and S3.
- Cost-effective: Pay only for actual consumed resources.

2.4 Use Cases

- Web Applications: Storing user profiles, preferences, and session data.
- Metadata Storage: Managing metadata for media files in content management systems.
- Dynamic/Unstructured Data: Logging, event tracking, IoT data storage.
- Low-Traffic Applications: Cost-effective for small workloads not needing complex databases.

2.5 Limitations

- Domain Size: Limited to 10 GB per domain.
- Attribute Size: Each value is limited to 1,024 bytes.

- Data Types: All data stored as strings. Number comparisons require zero-padded strings (e.g., '007' instead of '7').
- Scale: Largely superseded by DynamoDB for high-scale, production applications.

□ **Note:** *SimpleDB vs DynamoDB: SimpleDB is simpler, older, and has hard limits. DynamoDB is serverless, faster, and handles trillions of requests. Exam may ask you to compare them.*

3. Amazon DynamoDB

Amazon DynamoDB is a fully managed, serverless, NoSQL database service provided by AWS. It is a key-value and document database that delivers single-digit millisecond performance at any scale. AWS handles all infrastructure tasks — provisioning, replication, patching, and scaling.

3.1 Core Architecture Components

- **Tables:** Similar to relational DB tables but with a flexible schema.
- **Items:** Equivalent to rows/records. Each item is uniquely identifiable.
- **Attributes:** Equivalent to columns/fields.
- **Partition Key (Hash Key):** Mandatory attribute input to a hash function to determine which physical partition stores the data.
- **Sort Key (Range Key):** Optional second key. Items with the same Partition Key are sorted by Sort Key. Together they form a composite primary key.
- **Partitions:** Underlying storage mechanism. Data is automatically distributed across multiple physical partitions.
- **Replication:** Every item is replicated across 3 independent Availability Zones for high durability.

3.2 Working Mechanism

1. Request Routing: Every request (GetItem, PutItem) hits the Request Router first.
2. Routing Logic: Router uses Metadata to find which partition/node holds the data for a given Partition Key.
3. Authentication: IAM verifies identity.
4. Hashing: Partition Key is hashed to determine the storage node.
5. Write: Goes to the 'Leader' node; confirmed after quorum (majority) of replicas acknowledge.
6. Read: Eventually Consistent Read (any replica) or Strongly Consistent Read (from leader only).

3.3 Key Features

- Serverless: No servers to manage. Create a table and start using it immediately.
- Massive Scale: Handles 10+ trillion requests/day; peaks of 20 million requests/second.
- Global Tables: Multi-region, multi-active replication for globally distributed apps with local read/write performance.
- DynamoDB Accelerator (DAX): In-memory cache that reduces read response from milliseconds to microseconds.
- DynamoDB Streams: Captures time-ordered item-level changes (Insert/Update/Delete) — can trigger Lambda functions for real-time processing.
- Security: Data encrypted at rest by default. Fine-grained access control via IAM.

3.4 Limitations

- Item Size Limit: A single item cannot exceed 400 KB (including attribute names).
- No Joins: Cannot perform SQL-style joins or complex aggregations natively.
- Scan Performance: Scan reads every item — slow and costly for large datasets. Use Query instead.
- Hot Key Problem: Heavy access to a single partition key causes throttling (each partition: 3,000 RCUs / 1,000 WCUs limit).

3.5 Use Cases

- E-commerce: Shopping carts, session tracking (low latency critical).
- Gaming: Leaderboards, player states, matchmaking at massive scale.
- IoT: Ingesting high-velocity sensor data for real-time monitoring.
- Mobile Apps: Backend for apps requiring consistent performance regardless of user count.
- Ad Tech: User metadata retrieval in milliseconds for ad serving.

4. Amazon Storage System

Amazon Storage System is a suite of scalable, secure, and highly available cloud storage services by AWS. Rather than a single 'drive,' it is a collection of different storage architectures designed for different needs.

4.1 Key Concepts

- **Durability:** Probability that data will not be lost. Amazon S3 offers 99.999999999% (11 nines) durability.
- **Availability:** Percentage of time the storage service is accessible.
- **Scalability:** Ability to increase/decrease storage capacity instantly without hardware changes.

4.2 Types of Amazon Storage

Type	Service	Analogy	Best For
Object Storage	Amazon S3	Library with unique IDs for each book	Photos, videos, backups, data lakes
Block Storage	Amazon EBS	Virtual hard drive plugged into a PC	Databases, operating systems
File Storage	Amazon EFS	Shared network folder for an office	Content management, shared code

4.3 Working Mechanism

- Request: User/app sends an API call to store or retrieve data.
- Redundancy: S3 automatically replicates data across at least 3 AZs.
- Indexing: Unique identifier (Key) assigned to each data object.
- Retrieval: Metadata and keys used to pull data from the most efficient physical location.

4.4 Key Features

- Lifecycle Management: Automatically move data to cheaper tiers (e.g., S3 Glacier) after a set period.
- Encryption: Data encrypted at rest and in transit using AES-256.
- Versioning: Keep multiple versions of an object for protection against accidental deletion.
- Global Reach: Data storable in dozens of regions to reduce latency.

4.5 Use Cases

- Backup & Restore: Storing critical business data for disaster recovery.
- Data Lakes: Storing raw data for Big Data analytics and AI/ML training.
- Static Website Hosting: Serving HTML, CSS, JS files directly from S3.
- Content Distribution: Storing media delivered via Amazon CloudFront (CDN).

4.6 Limitations

- Latency: S3 (object storage) has higher latency than EBS (block storage) — not suitable for high-speed transactional DBs.

- Cost Complexity: Hidden costs from Data Transfer Out and API Request fees.
- Global Namespace: S3 bucket names must be globally unique (being addressed in newer AWS versions).

5. Cloud Computing Applications

5.1 Healthcare: ECG Analysis in the Cloud

ECG (Electrocardiogram) analysis traditionally required physical presence at a clinic. Moving to the cloud enables real-time heart monitoring, automated abnormality detection (arrhythmias), and global accessibility for doctors. The system follows an IoT-to-Cloud architecture.

Architecture Flow:

Layer	What Happens	AWS Service Used	Why
1. Data Acquisition (Edge)	Wearable sensor captures heart signals, sent via Bluetooth to smartphone	AWS IoT Core / Azure IoT Hub	Securely manages thousands of devices using lightweight MQTT protocol
2. Data Storage	Raw ECG data uploaded to cloud. Dual storage: Hot (recent) + Cold (historical)	Amazon DynamoDB (hot), Amazon S3 (cold)	DynamoDB for ms-speed real-time; S3 for cheap large-scale historical storage
3. Signal Pre-processing	Noise filtering (movement, electrode interference), signal normalization	AWS Lambda (serverless)	Scales instantly — 100 patients = 100 parallel Lambda instances. No idle cost.
4. AI Analysis	ML model (CNN) detects Atrial Fibrillation, Tachycardia, etc.	Amazon SageMaker	Managed platform to train & deploy medical ML models as an API
5. Alerting & Visualization	Doctor/patient notified if abnormality detected. Live ECG dashboard shown	Amazon SNS (alerts), AWS Amplify/AppSync (dashboard)	SNS delivers SMS/Email; AppSync syncs real-time graph to doctor's screen

Why Cloud for ECG?

- Scalability: Monitor 10 or 10,000 patients without buying new hardware.
- Accessibility: A specialist anywhere can analyze a patient's ECG in real-time.
- Accuracy: AI trained on millions of ECG samples worldwide is more accurate than local software.

5.2 Biology: Protein Structure Prediction

Protein Structure Prediction determines the 3D shape of a protein from its amino acid sequence. This is computationally massive (a single protein can fold into trillions of possible shapes). The cloud enables this by providing on-demand HPC resources. Accurate prediction is critical for drug design and biotechnology.

Proteins are chains of amino acids joined by peptide bonds. The rotational flexibility of the main chain creates multiple possible 3D structures (conformations). The goal is to find the most stable (lowest energy) conformation.

Cloud Pipeline for Protein Structure Prediction:

Step	Process	Cloud Technology	Why Cloud Matters
1. Sequence Ingestion	User uploads a .FASTA amino acid sequence via a web portal	AWS Amplify (scalable web interface)	No server management needed for the upload portal
2. Multiple Sequence Alignment (MSA)	Input sequence compared against massive genomic databases (UniProt, BFD) to find evolutionary relatives	Cloud Batch (CPU-intensive parallel search)	Spins up hundreds of CPU cores for parallel DB search, shuts down after — saves cost
3. 3D Structure Prediction	Aligned data fed into AlphaFold2 / RoseTTAFold deep learning model to predict atomic distances	Amazon SageMaker / Google Vertex AI (GPU)	Reduces prediction time from weeks on a laptop to minutes/hours using supercomputer-grade GPUs
4. Folding Refinement	Molecular Dynamics simulation (OpenMM) removes atomic clashes, finds minimum-energy state	Spot Instances (70-90% cheaper)	Discrete task — Spot Instances offer massive savings for non-critical batch jobs
5. Result Storage & Visualization	3D structure saved as .PDB / .mmCIF file. 3D viewer (Mol*) for interactive inspection	Amazon S3 + CloudFront	S3 for durable storage; CloudFront for fast global delivery of 3D model

Why Cloud for Biology?

- Parallelism: Predict 1,000 protein structures simultaneously by scaling out.
- Access to Data: Cloud providers host public datasets (Protein Data Bank) internally — no 500GB downloads.
- Collaborative Research: Scientists in different countries access the same VM or S3 bucket in real-time.

5.3 Geosciences: Satellite Image Processing

Cloud computing revolutionizes geosciences by providing massive storage and HPC power to process petabytes of satellite data. Use case: detecting urban expansion or deforestation in a region using satellite imagery (e.g., Sentinel-2, Landsat 8).

Processing Pipeline:

Step	Process	Cloud Technology	Why
1. Data Ingestion & Storage	Raw satellite images stored for parallel access	Amazon S3 / Google Cloud Storage (Object Storage)	High durability; fetch only specific tiles/bands needed — no full file download

Step	Process	Cloud Technology	Why
2. Pre-processing	Remove cloud cover, haze, sensor noise; clean data to reflect actual ground conditions	AWS Lambda / Cloud Serverless (parallel tiles)	Break 1GB image into 1,000 chunks; process all simultaneously in seconds
3. Feature Extraction	Calculate NDVI (Normalized Difference Vegetation Index) to distinguish water, vegetation, concrete	Google Earth Engine (MapReduce)	Hosts public satellite catalog; runs NDVI across thousands of images in parallel via JS/Python
4. Classification	ML model labels each pixel as 'Forest', 'Urban', 'Water', etc.	Amazon SageMaker / Azure ML (Random Forest, CNN)	Scale GPU instances to train on millions of pixels; deploy as endpoint for new images
5. Visualization & Analysis	Change Map shows deforestation/urbanization over time	Microsoft Planetary Computer / Tableau / Power BI	Interactive Jupyter Notebooks; export statistical reports

□ **Note:** $NDVI = (NIR - Red) / (NIR + Red)$. Values near 1 indicate dense vegetation; values near 0 indicate bare soil or concrete.

5.4 Business Applications: CRM and ERP

CRM (Customer Relationship Management) and ERP (Enterprise Resource Planning) are core business software systems moved to the cloud for scalability and accessibility. Integrating them gives businesses a unified view of a customer's journey — from first ad click to final shipment.

Cloud CRM & ERP Integration Steps:

Step	Process	Cloud Technology	Why
1. Centralization	Move fragmented data (Excel, local DBs) to a single source of truth	SaaS — Salesforce, NetSuite	Eliminates data silos; all teams see the same real-time data
2. Secure Access	Identity management for safe login from anywhere	IAM (Identity & Access Management)	Role-based access — a sales rep sees customer data, not payroll
3. Workflow Automation	Auto-generate an invoice when a deal is marked 'Closed' in CRM	Serverless Functions (AWS Lambda / Azure Functions)	Triggers code only when needed — no idle server cost
4. Predictive Insights	Analyze sales trends to predict stock requirements for next month	Cloud AI/ML — AWS SageMaker	Processes massive historical data to find patterns humans would miss

5.5 Consumer Applications: Social Networking

Social media platforms handle bursty traffic (millions logging on during a cricket match) and store vast amounts of unstructured data (videos, photos).

Step	Process	Cloud Technology	Why
1. Content Delivery	User in Tokyo views video uploaded in New York without lag	CDN (Content Delivery Network)	CDNs cache content on servers geographically close to users — reduces latency
2. Massive Storage	Host billions of photos, Stories, and media	Amazon S3 / Google Cloud Storage (Object Storage)	Traditional file systems would crash; S3 is 'infinite' and cheap for cold data (old posts)
3. Traffic Management	Distribute millions of login requests across thousands of servers	Elastic Load Balancers	Prevents server overload; reroutes traffic instantly if a server fails
4. Real-time Feeds	Live updates, likes, comments appearing instantly	NoSQL Databases (MongoDB / DynamoDB)	Handles unstructured data and writes millions of updates per second

6. Google Cloud Application: Google App Engine (GAE)

Google App Engine (GAE) is a Platform as a Service (PaaS) that allows developers to deploy applications without worrying about servers, OS patching, or infrastructure management. It is serverless — Google automatically scales the app up or down based on traffic.

6.1 How Google App Engine Works

Step 1 — Writing the Code: Develop your application in a supported language: Python, Java, Node.js, Go, etc. (e.g., a Python Flask app with POST to save messages and GET to retrieve them).

Step 2 — Configuration (app.yaml): Instead of configuring a web server (Apache/Nginx), you create an app.yaml file specifying the runtime (e.g., Python 3.12) and scaling behaviour. This is the bridge between your code and Google's infrastructure.

Step 3 — Persistent Data Storage: App Engine is stateless — servers forget data on restart. Google Cloud Firestore (Datastore mode) is used as the NoSQL document database for permanent storage. It is also serverless and scales perfectly with App Engine.

Step 4 — Deployment: Use Google Cloud CLI with the command 'gcloud app deploy'. Google builds a container image, deploys it to a secure sandbox, and assigns a URL (e.g., your-project-id.appspot.com).

Step 5 — Traffic Splitting & Versioning: Deploy Version 2 of your app, route 10% of users to it, and 90% to the old version. Slide to 100% if stable. Instant rollback to Version 1 if Version 2 crashes.

6.2 Key Features of Google App Engine

- Zero Infrastructure Management: No SSH into VMs, no OS updates — Google manages the infrastructure.
- Automatic Scaling (Zero to Hero): Scales to zero instances when idle (pay \$0). Spins up hundreds of instances in seconds if 100,000 users click at once.
- Integrated Security: Built-in SSL/HTTPS. Google manages certificates automatically.
- Versioning & Traffic Splitting: Test new versions with canary deployments before full rollout.
- Multiple Languages Supported: Python, Java, Node.js, Go, PHP, Ruby.

□ **Note:** GAE is a PaaS product. Key exam distinction: IaaS (you manage the VM), PaaS (you manage only the code), SaaS (you just use the app). GAE = PaaS.

7. Overview of OpenStack Architecture

OpenStack is an open-source cloud operating system that allows organizations to build and manage their own private or public cloud infrastructure using commodity (standard, off-the-shelf) hardware. It abstracts physical hardware into a flexible, virtualized pool of resources accessible via APIs.

7.1 OpenStack as a Cloud Operating System

OpenStack sits between the physical hardware and end-user applications, just as a traditional OS sits between hardware and user programs. It abstracts complexity through a service-oriented design — each major function is a separate service.

7.2 Layers of OpenStack Architecture

Layer 1 — Standard Hardware (Bottom)

- Physical servers (x86), storage arrays, and network switches.
- OpenStack runs on commodity hardware — no expensive specialized equipment needed.
- These physical boxes are turned into a flexible, virtualized resource pool.

Layer 2 — OpenStack Shared Services (Integration Layer)

- Acts as the integration backbone for the entire platform.
- Keystone (Identity Service): Handles authentication and issues tokens. Every service call must be verified here first.
- Message Bus (RabbitMQ): Enables services to communicate with each other asynchronously.
- Service Catalog: Keeps track of where every service is located.

Layer 3 — Core Resources (The Cloud Operating System)

Resource Type	OpenStack Component	Function
Compute	Nova	Manages the lifecycle of Virtual Machines (VMs). Handles scheduling — deciding which physical server hosts a new VM.
Network	Neutron	Provides Network-as-a-Service. Creates virtual switches, routers, and firewalls for cloud instances.
Block Storage	Cinder	Provides virtual hard drives (block volumes) for VMs — similar to AWS EBS.
Object Storage	Swift	Provides scalable object storage for files/images — similar to AWS S3.
Image Service	Glance	Stores and retrieves VM disk images (like AWS AMIs). Used by Nova when launching VMs.
Identity Service	Keystone	Authentication and authorization for all OpenStack services.

Layer 4 — APIs and Dashboard

- Dashboard (Horizon): Web-based GUI (graphical user interface). Allows admins to launch VMs, manage networks, and monitor resource usage via browser.

- APIs: Every service in OpenStack exposes a RESTful API. Whether you're a human using a script or an application requesting resources, everything goes through these standardized APIs.

Layer 5 — Applications (Top)

- The consumer layer — actual software running on the cloud: web servers, databases, AI models, Big Data jobs.
- These applications are unaware of the physical hardware below.
- They scale up/down by sending API requests to the cloud OS.

7.3 OpenStack Request Flow (Important for Exams)

Example: An application needs more storage.

- 1. Application sends an API request to the Cloud Operating System.
- 2. Shared Services (Keystone) verify the request is authorized.
- 3. Request is routed to the appropriate service (e.g., Cinder for block storage).
- 4. OpenStack allocates the required portion of Standard Hardware.
- 5. The resource is delivered virtually to the application.

□ **Note:** OpenStack is the open-source alternative to AWS/Azure/GCP for building private clouds. Exam may ask: What is OpenStack? Why use it? Compare it with AWS.

8. Quick Revision — Important Comparisons

8.1 SimpleDB vs DynamoDB

Feature	Amazon SimpleDB	Amazon DynamoDB
Type	Non-relational, distributed DB	Serverless, NoSQL (key-value + document)
Scale	Small workloads (10GB limit per domain)	Massive scale — 10 trillion requests/day
Performance	Good for simple queries	Single-digit millisecond at any scale
Schema	Schema-less, dynamic attributes	Schema-less, flexible
Indexing	Auto-indexes all attributes	Primary key + GSI/LSI for additional indexes
Data Types	All data stored as strings	Rich types — strings, numbers, binary, lists, maps
Status	Legacy service — rarely used for new projects	Recommended for modern, high-scale applications

8.2 S3 vs EBS vs EFS

Feature	Amazon S3	Amazon EBS	Amazon EFS
Type	Object Storage	Block Storage	File Storage
Access	Via HTTP/HTTPS API	Attached to single EC2 instance	Mounted by multiple EC2 instances (NFS)
Durability	99.999999999%	99.999%	High (Multi-AZ)
Best For	Images, videos, backups, data lakes	OS, databases	Shared content, distributed apps
Protocol	REST API	Block device	NFS

8.3 Cloud Service Model Comparison

Model	You Manage	Provider Manages	Example
IaaS	OS, runtime, apps, data	Hardware, network, hypervisor	Amazon EC2
PaaS	Applications and data only	OS, runtime, hardware	Google App Engine
SaaS	Nothing (just use the app)	Everything	Salesforce CRM, Gmail

8.4 OpenStack Core Services Summary

Service Name	Component	Equivalent AWS Service
Compute	Nova	Amazon EC2
Networking	Neutron	Amazon VPC

Service Name	Component	Equivalent AWS Service
Block Storage	Cinder	Amazon EBS
Object Storage	Swift	Amazon S3
Image Service	Glance	AWS AMI (AMI Catalog)
Identity	Keystone	AWS IAM
Dashboard	Horizon	AWS Management Console

9. Probable Exam Questions & Key Points

Short Answer / Definition Questions

- Define AWS and list its key characteristics. (On-demand, scalable, pay-as-you-go)
- What is Amazon SimpleDB? List its features and limitations.
- Explain DynamoDB architecture with its core components (Tables, Items, Attributes, Partition Key, Sort Key).
- What is Amazon S3? Explain its components (Buckets, Objects, Keys, Storage Classes).
- What is Google App Engine? How does it differ from EC2? (PaaS vs IaaS)
- Define OpenStack. What role does Keystone play?
- What is NDVI? Where is it used? (Satellite Image Processing application)

Long Answer / Application Questions

- Explain how cloud computing is used for ECG Analysis in Healthcare. Describe each layer of the architecture.
- Describe the cloud pipeline for Protein Structure Prediction in Biology. Why is GPU important here?
- Explain cloud-based Satellite Image Processing for deforestation detection. Name the tools used at each step.
- With a neat diagram/description, explain OpenStack architecture — from hardware to applications.
- Compare SimpleDB and DynamoDB. When would you choose each?
- Explain how Social Networking applications use cloud computing. Give examples of cloud services used.

Key Definitions to Memorize

- EC2: Resizable virtual servers in the AWS cloud.
- S3: Object storage for any amount of data, accessed via HTTP API.
- Lambda: Serverless function execution — triggered by events, scales automatically.
- DynamoDB: Fully managed serverless NoSQL database with single-digit ms performance.
- SimpleDB: Schema-less, distributed DB limited to 10GB/domain; auto-indexes all attributes.
- VPC: Virtual Private Cloud — isolated network section in AWS.
- IAM: Identity and Access Management — controls who can access what in AWS.
- App Engine: Google's PaaS — deploy code without managing any servers.
- OpenStack: Open-source private cloud OS managing compute, storage, and network via APIs.
- NDVI: Normalized Difference Vegetation Index — used in satellite imagery to detect vegetation.
- MSA: Multiple Sequence Alignment — comparing protein sequences to find evolutionary patterns.
- AlphaFold: Deep learning model by DeepMind for 3D protein structure prediction.